BACSE202 Database Systems

REVIEW-1

Centralized Blood Bank & Emergency Routing

Team Members:

- Aarnaa 25BCE0675
- Aayushi 25BCE0646
- Ankita 25BCE0649
- Muskan 25BCE2039

Guided By:

Dr. Somasundaram R

Problem Statement

Existing hospital blood banks often maintain their blood inventory independently, making it difficult to quickly identify compatible blood units available at nearby hospitals during emergencies.

This results in:

- Delays in locating required blood during critical situations.
- Difficulty in identifying compatible and available blood units.
- Blood-unit wastage when units expire without timely allocation.
- Lack of centralized visibility of blood inventory and emergency requests.

Our Proposed Solution

A centralized Blood Bank & Emergency Routing System that manages blood inventory, expiry information, hospital requests and blood-unit allocation through a common database.

Scope and Users

The system will:

- Maintain donor and blood-bag records by tracking blood-group, Rh factor, collection date and expiry date.
- Manage emergency blood requests.
- Match requests with compatible available blood units.
- Prioritize these requests as **Critical, Urgent or Routine.**
- Track allocation and blood-bag status.
- Identify low-stock and expired/near-expiry blood units.

Users

- **Hospital / Blood Bank Staff** – manage blood inventory and requests.
- **Emergency / Medical Staff** – submit and monitor blood requirements.
- **System Administrator** – manage hospitals, donors and system records.
- **Donors** – provide and maintain donor information.

Functional Requirements

1. Management of Hospitals

- Register and maintain hospital details, location.

2. Donor Management

- Store donor details, blood group, Rh factor and donation information.

3. Blood Inventory Management

- Register individual blood bags.
- Record collection and expiry dates.
- Track current blood-bag status.

4. Emergency Request Management

- Allow hospitals to submit blood requests.
- Record required blood group, Rh factor, units and priority.
- Track request status.

5. Blood Matching & Allocation

- Identify compatible available blood units.
- Allocate suitable units to requests.
- Process higher-priority emergency requests first.

6. Expiry & Inventory Monitoring

- Detect expired or near-expiry blood units.
- Generate low-stock/expiry warnings.

7. Status Tracking

- Track blood bags through states such as Available → Reserved → Dispatched / Expired.
- Maintain a history of status changes.

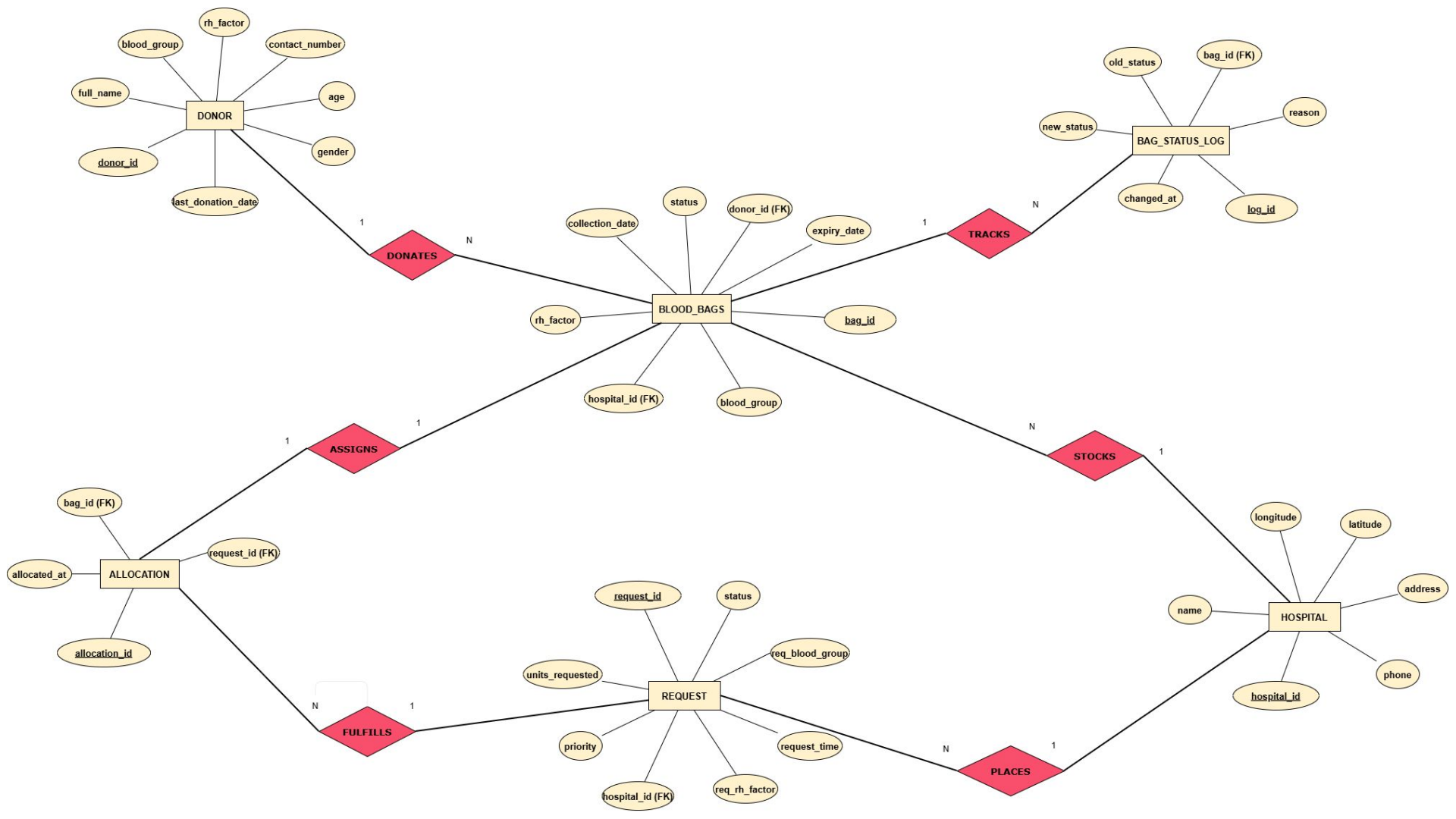
ER / EER Diagram

The ER model structures a centralized blood-routing system across six normalized entities. It tracks blood units from collection (DONOR to BLOOD_BAGS) to regional inventory storage within HOSPITALS. Emergency orders placed by hospitals (REQUEST) are routed through an associative entity (ALLOCATION), which assigns non-expired compatible bags under strict one-to-one constraints to prevent double-allocation. All life-cycle state transitions are captured in BAG_STATUS_LOG to guarantee audit traceability, concurrency safety, and full relational integrity.

Full ER Diagram

View the detailed schema diagram including cardinality constraints, primary keys, and foreign key relationships online.

https://drive.google.com/file/d/1iCXhpcqxF2g7NqSma5-pl1KVzuD1Nqz7/view?usp=sharing


DONOR
blood_group
rh_factor
contact_number
full_name
age
donor_id
gender
last_donation_date
DONATES
1
N
BLOOD_BAGS
collection_date
status
donor_id (FK)
expiry_date
rh_factor
bag_id
hospital_id (FK)
blood_group
TRACKS
1
N
BAG_STATUS_LOG
old_status
bag_id (FK)
reason
new_status
changed_at
log_id
ASSIGNS
1
1
ALLOCATION
bag_id (FK)
request_id (FK)
allocated_at
allocation_id
STOCKS
N
1
HOSPITAL
longitude
latitude
address
name
phone
hospital_id
FULFILLS
N
1
REQUEST
request_id
status
req_blood_group
units_requested
priority
request_time
hospital_id (FK)
req_rh_factor
PLACES
N
1

ER Diagram to Relational Model

Mapping Rules Applied

1. Entity → Table

Each of the 6 entities becomes a table: DONOR, HOSPITAL, BLOOD_BAGS, REQUEST, ALLOCATION, BAG_STATUS_LOG.

2. Attribute → Column

Every attribute becomes a column; the entity ID is the Primary Key (PK).

3. 1 : N → FK on N side

The PK of the "1" side is added as a Foreign Key in the "N" side table.

4. 1 : 1 → FK + UNIQUE

ASSIGNS: BAG_ID in ALLOCATION is UNIQUE, so a bag can never be double-allocated.

5. Associative entity

ALLOCATION links REQUEST and BLOOD_BAGS and holds BAG_ID and REQUEST_ID as FKs.

Relationship Mapping

Relation ship	Cardinality	Mapping in relational model
DONATES	DONOR 1 : N BLOOD_BAGS	DONOR_ID as FK in BLOOD_BAGS
STOCKS	HOSPITAL 1 : N BLOOD_BAGS	HOSPITAL_ID as FK in BLOOD_BAGS
TRACKS	BLOOD_BAGS 1 : N BAG_STATUS_LOG	BAG_ID as FK in BAG_STATUS_LOG
PLACES	HOSPITAL 1 : N REQUEST	HOSPITAL_ID as FK in REQUEST
ASSIGNS	BLOOD_BAGS 1 : 1 ALLOCATION	BAG_ID as FK in ALLOCATION, UNIQUE
FULFILLS	REQUEST 1 : N ALLOCATION	REQUEST_ID as FK in ALLOCATION

Relational Model — Table Schema

Primary keys underlined (PK); foreign keys marked FK → parent table

DONOR *Registered donors*

- **DONOR_ID** PK
- FULL_NAME
- AGE
- GENDER
- BLOOD_GROUP
- RH_FACTOR
- CONTACT_NUMBER
- LAST_DONATION_DATE

HOSPITAL *Hospital contact and geo-location*

- **HOSPITAL_ID** PK
- NAME
- ADDRESS
- LATITUDE
- LONGITUDE
- PHONE

BLOOD_BAGS *Live inventory units*

- **BAG_ID** PK
- BLOOD_GROUP
- RH_FACTOR
- COLLECTION_DATE
- EXPIRY_DATE
- STATUS
- **DONOR_ID** FK → DONOR
- **HOSPITAL_ID** FK → HOSPITAL

REQUEST *Emergency requests by hospitals*

- **REQUEST_ID** PK
- REQ_BLOOD_GROUP
- REQ_RH_FACTOR
- UNITS_REQUESTED
- PRIORITY
- STATUS
- REQUEST_TIME
- **HOSPITAL_ID** FK → HOSPITAL

ALLOCATION *Associative: request to bag*

- **ALLOCATION_ID** PK
- **BAG_ID** FK → BLOOD_BAGS (UNIQUE)
- **REQUEST_ID** FK → REQUEST
- ALLOCATED_AT

BAG_STATUS_LOG *Audit trail of status changes*

- **LOG_ID** PK
- **BAG_ID** FK → BLOOD_BAGS
- OLD_STATUS
- NEW_STATUS
- CHANGED_AT
- REASON

Normalization - Functional Dependencies

R = all attributes of the six entities in one flat relation. Dependencies read from the ER diagram (donor_ and bag_ prefixes separate the two blood-group pairs):

Determinant	Determines
request_id	hospital_id, req_blood_group, req_rh_factor, units_requested, priority, request_status, request_time
hospital_id	name, address, latitude, longitude, phone
bag_id	donor_id, stock_hospital_id, bag_blood_group, bag_rh_factor, collection_date, expiry_date, bag_status
donor_id	full_name, age, gender, donor_blood_group, donor_rh_factor, contact_number, last_donation_date
allocation_id	request_id, bag_id, allocated_at
log_id	bag_id, old_status, new_status, changed_at, reason

First Normal Form (1NF)

- A request is fulfilled by several bags, so bag columns repeat inside one request row.
- A bag has many status-log entries, so log columns repeat inside one bag.

Before (repeating groups)

request_id	bag_id	log_id
R1	B1, B2	L1, L2, L3

After 1NF (atomic cells)

request_id	bag_id	log_id
R1	B1	L1
R1	B1	L2
R1	B2	L3

Justification: every cell holds one atomic value, repeating groups are gone, and the primary key is (request_id, bag_id, log_id).

Second Normal Form (2NF)

1NF key is (request_id, bag_id, log_id). Attributes that depend on only part of it are moved out:

Partial dependency	Moved to
request_id → request columns and requesting-hospital columns	REQUEST_R
bag_id → bag columns and donor columns	BAG_R
(request_id, bag_id) → allocation_id, allocated_at	ALLOCATION
log_id → bag_id, old_status, new_status, changed_at, reason	BAG_STATUS_LOG

Justification: In each new table every non-key attribute depends on the whole key. Hospital and donor columns still ride along inside REQUEST_R and BAG_R, which 3NF removes.

Third Normal Form (3NF)

A non-key attribute must not depend on another non-key attribute. Three transitive dependencies remain:

Transitive dependency	Fix
request_id → hospital_id → name, address, latitude, longitude, phone	HOSPITAL
bag_id → stock_hospital_id → hospital columns	HOSPITAL
bag_id → donor_id → full_name, age, gender, contact_number, last_donation_date, donor blood group	DONOR

Anomalies removed: Hospital address is stored once instead of in every request row, a donor can be recorded before any bag exists.

Allocation check: allocation_id and bag_id (UNIQUE) are both candidate keys, so every determinant is a key.

Final 3NF Schema

Table	Primary key	Foreign keys
DONOR	donor_id	–
HOSPITAL	hospital_id	–
BLOOD_BAGS	bag_id	donor_id, hospital_id
REQUEST	request_id	hospital_id
ALLOCATION	allocation_id	bag_id (UNIQUE), request_id
BAG_STATUS_LOG	log_id	bag_id

Justification: The six tables match the ER-to-relational mapping, and each satisfies 1NF, 2NF and 3NF.